→ Math flow

text = "I Love AI"

1) input

S = "I Love AI"

- A neural network cannot directly perform numerical operation on text, so we first convert the text into token ID's

2) operation - Tokenization.

- Let T represent the tokenizer

$T(s) = [17, 245, 901]$ → these are random no for Nodust for example. the actual ID's depend on the tokenizer used by your LLM.

3) 1D Tensor

- Represent the token IDs as a 1D vector

$x = [17, 245, 901]$

- the shape is

$\text{Shape}(x) = (3,)$.

4) Embedding

- Suppose the embedding dimension is $d = 768$
- The embedding layer contains a learnable weight Matrix

$$W_E \in R^{50257 \times 768}$$

→ these again depends on
1. the tokenizer one chooses
2. the dim they think fits the best.

- we select the corresponding rows.

$$e_{17} = W_E[17, :]$$
$$e_{245} = W_E[245, :]$$
$$e_{901} = W_E[901, :]$$

- output

$$E = \begin{bmatrix} e_{17} \\ e_{245} \\ e_{901} \end{bmatrix}$$

→ each token becomes a vector containing 768 numbers.

output shape = (3, 768).

5) positional Embedding

Let's say: context-length = 1024

$d_{model} = 768$.

So the embedding Matrix is

$$W_p \in R^{1024 \times 768}$$

- There are three tokens, so the positions are:

$$P_{IDs} = [0, 1, 2]$$

- operation

$$P_0 = W_P[0, :]$$
$$P_1 = W_P[1, :]$$
$$P_2 = W_P[2, :]$$

- output - positional Embeddings.

$$E_{pos} = \begin{bmatrix} P_0 \\ P_1 \\ P_2 \end{bmatrix}$$

- output shape

$$(3, 768)$$

6) Token Embedding + positional Embedding

$$E_{token} = \begin{bmatrix} e_{17.1}, e_{17.2} & \cdots & e_{17.768} \\ e_{245.1}, e_{245.2} & \cdots & e_{245.768} \\ e_{901.1}, e_{901.2} & \cdots & e_{901.768} \end{bmatrix}_{(3, 768)}$$

$$E_{pos} = \begin{bmatrix} P_{0.1} & P_{0.2} & \cdots & P_{0.768} \\ P_{1.1} & P_{1.2} & \cdots & P_{1.768} \\ P_{2.1} & P_{2.2} & \cdots & P_{2.768} \end{bmatrix}_{(3, 768)}$$

$$E_{input} = E_{token} + E_{pos}$$

$$E_{input} = \begin{bmatrix} e_{17,1} + p_{0,1} & \cdots & e_{17,768} + p_{0,768} \\ e_{245,1} + p_{1,1} & \cdots & e_{245,1} + p_{1,768} \\ e_{901,1} + p_{2,1} & \cdots & e_{901,1} + p_{2,768} \end{bmatrix}$$

$$E_{input} = \begin{bmatrix} e_{17} + p_0 \\ e_{245} + p_1 \\ e_{901} + p_2 \end{bmatrix}_{(3,768)}$$

7) Embedding dropout

Let's Say $\boxed{p = 0.1}$

- Here p is the dropout probability, meaning each individual embedding value has a 10% probability of being dropped during training.
- During training, dropout generates a random mask M, whose entries are either 0 or 1

For each element

$$M_{i,j} \approx \text{Bernoulli}(1-p)$$

Since $p = 0.1$

$$P(M_{i,j} = 0) = 0.1$$

$$P(M_{i,j} = 1) = 0.9$$

- every surviving value is also divided by $1-p$
- the operation

$$E_{drop} = \frac{M \odot E_{input}}{1-p}$$

$$E_{drop} = \frac{M \odot E_{input}}{0.9}$$

=> Small example of dropout

- $e = [2, 4, 6, 8]$
- $M = [1, 0, 1, 1]$
- $M \odot e = [2, 0, 6, 8]$
- $E_{drop} = \left[\frac{2}{0.9}, 0, \frac{6}{0.9}, \frac{8}{0.9}\right]$

$E_{drop} = [2.22, 0, 6.67, 8.89]$

- Output - Dropped embedding

$$E_{drop} = \begin{bmatrix} z_{1,1} & z_{1,2} & \cdots & z_{1,768} \\ z_{2,1} & z_{2,2} & \cdots & z_{2,768} \\ z_{3,1} & z_{3,2} & \cdots & z_{3,768} \end{bmatrix}$$

$(3, 768)$

- the shape does not change

$$(3, 768) \xrightarrow{\text{dropout}} (3, 768)$$

- In actual batched model, the corresponding flow is

$$(1, 3, 768) \xrightarrow{\text{dropout}} (1, 3, 768).$$

8) Layer Normalization

$$X = E_{drop}$$

$$X \in R^{3 \times 768}$$

8.1) First preserve the input for shortcut

$$\boxed{S = X}$$

—

- LayerNorm normalizes the embedding features of each token independently.

- We calculate the mean and variance across the 768 features of each token separately.

Step-1) Calculate the mean.

- for token i, the mean of its 768 features is

$$\mu_i = \frac{1}{768} \sum_{j=1}^{768} x_{i,j}$$

for our three tokens

$$\mu_1 = \frac{1}{768} \sum_{j=1}^{768} x_{1,j}$$

$$\mu_2 = \frac{1}{768} \sum_{j=1}^{768} x_{2,j}$$

$$\mu_3 = \frac{1}{768} \sum_{j=1}^{768} x_{3,j}$$

Step-2) Calculate the Variance

- for each token, how far its features are from that token's mean

$$\sigma_i^2 = \frac{1}{768} \sum_{j=1}^{768} (x_{i,j} - \mu_i)^2$$

- Therefore

$$\sigma_1^2, \sigma_2^2, \sigma_3^2$$

We now have one variance for each token

Step-3: Normalize each feature.

- Subtract the mean and divide by the standard deviation, with a small numerical stabilizer ε:

$$x_{i,j} = \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

- Let's say $\varepsilon = 10^{-5}$

So for the first token

$$x_{1,j} = \frac{x_{1,j} - \mu_1}{\sqrt{\sigma_1^2 + 10^{-5}}}$$

- like this we get $x_{2,j}$, $x_{3,j}$

- Now we have

$$x_{1,j}, x_{2,j}, x_{3,j}$$

Step-4: apply scale and shift

$$\gamma, \beta \in \mathbb{R}^{768}$$

- they are initialized to ones (scale) and zeros (shift). respectively, and are learned during training.

- We cal the final normalized output using

$$y_{i,j} = \gamma_j \cdot x_{i,j} + \beta_j$$

- Output — Normalized embeddings

$$Y = \begin{bmatrix} y_{1,1} & y_{1,2} & \cdots & y_{1,768} \\ y_{2,1} & y_{2,2} & \cdots & y_{2,768} \\ y_{3,1} & y_{3,2} & \cdots & y_{3,768} \end{bmatrix}$$

$$Y \in R^{3 \times 768}$$

9) generate queries, keys and values (Q, K, V)

Let's say

$$d_{model} = 768, \quad h = 2, \quad L = 1$$

h = heads
L = Layer which is no of transformer blocks

$$X_N = Y = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,768} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,768} \\ x_{3,1} & x_{3,2} & \cdots & x_{3,768} \end{bmatrix}$$

- for the actual model, the batch dimension is retained, So the shape is (1, 3, 768)

where

$$B = 1, \quad T = 3, \quad d_{model} = 768$$

-> operation — Three linear projections.

- The attention mechanism creates three different representations of the same input

i) Query (Q): Represents what a token is looking for when matching against other tokens

ii) Key (K): Represents the features used to match tokens against queries

(iii) Value (V): Contains the information that will be combined according to the attention scores.

- Each layer has its own learned weight matrix. So the three projections generally produce different numerical values even though they receive the same input.

Step - 1: Query matrix.

Mathematical operation:

$$Q = X_N W_Q^T$$

where $W_Q \in 768 \times 768$ } -> in-features = 768, out-features = 768

- Now calculate the dim

$$(1 \times 3 \times 768) \quad (768 \times 768)$$

- therefore

$$Q \in R^{1 \times 3 \times 768}$$

- It contains three query vectors

$$Q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

Step-2) key matrix

- We project the same input using a different learned W matrix

$$K = X_N W_K^T \qquad W_K \in R^{768 \times 768}$$

- Calculate the dim

$$(1, 3, 768) \quad (768, 768).$$

- therefore

$$K \in R^{1 \times 3 \times 768}$$

- It contains three key vectors

$$K = \begin{bmatrix} K_1 \\ K_2 \\ K_3 \end{bmatrix}$$

Step-3: Value matrix

$$V = X_N W^T_V \qquad W_V \in R^{768 \times 768}$$

- Cal the dim

$$(1, 3, 768) \quad (768, 768)$$

- therefore

$$V \in R^{1 \times 3 \times 768}$$

- It contains three value vectors

$$V = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

So

Output:

$$Q, K, V \in R^{1 \times 3 \times 768}$$

10) Now Splitting into two heads.

- the dimension assigned to each head is

$$d_{head} = \frac{d_{model}}{h}$$

Substitute our values:

$$d_{head} = \frac{768}{2}$$

$$d_{head} = 384$$

- We will split each of Q, K and V into two heads and arrange the dimensions consistently

$(1, 3, 768) \longrightarrow (1, 2, 3, 384)$

Before splitting — after splitting and Transposing.

- operation - Reshape Q, K, V

$Q, K, V : (1, 3, 768) \longrightarrow (1, 3, 2, 384)$

- operation - B - transpose head and sequence dim
- the reshaped tensor has dimensions in this order

(B, T, h, d_{head}).

- for calculating attention conveniently, we want the heads dimension to come before the sequence dimension

(B, h, T, d_{head})

- Therefore we transpose dim 1 and 2 in zero-based tensor indexing

$(1, 3, 2, 384) \longrightarrow (1, 2, 3, 384)$

this applies to all three

$Q_{heads}, K_{heads}, V_{heads} \in \mathbb{R}^{1 \times 2 \times 3 \times 384}$

11. Calculating Attention Scores

- Now we will calculate how strongly each token matches the other tokens using query and key tensors.

- Query, key tensors are

$$Q_{head}, K_{head} \in R^{1 \times 2 \times 3 \times 384}$$

- we want to compare each token's query vector with every token's key vector, separately for each head.

- operation - A: Transpose the key Tensor.

- The attention score calculation uses the transpose of the key tensor's last two dimensions.

$$K^T_{heads} : (1, 2, 3, 384) \rightarrow (1, 2, 384, 3)$$

$$\boxed{K^T_{heads} \in R^{1 \times 2 \times 384 \times 3}}$$

- operation - B: Multiply Queries by transposed keys

$$S = Q \cdot K^T$$

$$(1, 2, 3, 384) @ (1, 2, 384, 3).$$

- the two inner dim are both 384. So the Matmx multiplication is valid.
- the output dimensions are

$$S \in R^{1\times 2\times 3\times 3}$$

- Head 2 generates its own 3×3 matrix

— Operation C — Scale the attention scores

- If we used the dot product directly, their magnitudes could become large as the feature dimension grows.
- Attention therefore scales the scores by the sqrt of head dimension.
- formula:

$$S_{scaled} = \frac{Q_{heads} \cdot K^{T}_{heads}}{\sqrt{d_{heads}}}$$

- $d_{head} = 384$

$\sqrt{384} = 19.596$.

- $S_{scaled} = \frac{Q_{heads} K^{T}_{heads}}{19.596}$

- Therefore,

$$S_{scaled} \in R^{1 \times 2 \times 3 \times 3}$$

12) Apply the Causal Attention Mask.

- Now we need to make sure our language model cannot look at future tokens when predicting the next token.

— operation - A Construct the causal mask.

- we need to allow each token to attend to it self and earlier tokens, while preventing it from attending to later tokens.
- For three tokens, the causal mask is

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Read each row as follows:

- Row 1 - I : can attend only to I
- Row 2 - love! can attend to I and love
- Row 3 - AI : can attend to I, love, and AI

- using $-\infty$ because the next operation will be Softmax. A score of $-\infty$ receives a probability of zero after softmax.

· The mask has shape

$$M \in R^{3\times 3}$$

· M is shared across both attention heads and the single sequence in our batch.

— operation-B : Add the causal mask to the scores.

$$S_{masked} = S_{scaled} + M$$

· For each head this means

$$\begin{bmatrix} S_{11} & S_{12} & S_{13} \\ S_{21} & S_{22} & S_{23} \\ S_{31} & S_{32} & S_{33} \end{bmatrix} + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

· produces :

$$\begin{bmatrix} S_{11} & -\infty & -\infty \\ S_{21} & S_{22} & -\infty \\ S_{31} & S_{32} & S_{33} \end{bmatrix}$$

$$S_{masked} \in R^{1\times 2\times 3\times 3}$$

13) Applying Softmax.

· We'll apply Softmax along the final dimension so each token's attention scores become probabilities that sum to 1.

• for example: the first row will assign probability 1 to I and probability 0 to both future tokens

– operation - Softmax.

$$A = \text{Softmax}(S_{masked}, dim = -1)$$

• for each row

$$a_{ij} = \frac{e^{S_{ij}}}{\sum_{k=1}^{3} e^{S_{ik}}}$$

• the masked positions are $-\infty$, so their attention weights become zero.

– output

$$A \in \mathbb{R}^{1 \times 2 \times 3 \times 3}$$

14) Attention Dropout

– operation - Apply Dropout

$$A_{drop} = \text{Dropout}(A)$$

$$A_{drop} = \frac{D \odot A}{1 - P}$$

- Here D is the random dropout mask.
- $\odot$ means element wise multiplication

- Output

$$A_{drop} \in \mathbb{R}^{1 \times 2 \times 3 \times 3}$$

- Dropout changes the numeric values, but not the tensor shape

15) Multiply attention weights by values.

- the operation that actually combines information from different tokens using the attention weights.

- We have

$$A_{drop} \in \mathbb{R}^{1 \times 2 \times 3 \times 3}$$

$$V \in \mathbb{R}^{1 \times 2 \times 3 \times 384}$$

- operation - Matrix Multiplication.

$$\boxed{C = A_{drop} V}$$

- Substitute the tensor shapes

$$(1, 2, 3, 3) \; @ \; (1, 2, 3, 384).$$

- the last two dim perform matrix multiplication separately for each batch and head

$$(3 \times 3)(3 \times 384).$$

Therefore

$$C \in R^{1 \times 2 \times 3 \times 384}$$

NOTE:

- for each head, the attention weights determine how much information to combine from each token's value vector.
- For a particular head, the context vector for token i is

$$c_i = \sum_{j=1}^{3} a_{ij} v_j$$

—

- output:

$$C \in R^{1 \times 2 \times 3 \times 384}.$$

- Now we have a 384 dimensional context vector for each of the three tokens in each of the two heads.

16) Transpose the context vector

- Operation - Transpose dim 1 and 2

$(1, 2, 3, 384) \longrightarrow (1, 3, 2, 384)$.

- The resulting tensor has dim ordered

(B, T, h, d_{head})

- we did it because we want the two heads representations for each token to sit next to each other in the tensor layout, ready to be combined into one 768-dim vector per token

- output $C_{Transposed} \in \mathbb{R}^{1 \times 3 \times 2 \times 384}$

17) Concatenate the Two Attention Heads.

- it brings two attention heads together into a single representation for each token.

- Operation # - Make the tensor contiguous.

- The previous transpose changed the tensor's dim layout. In pytorch a transpose can produce a tensor whose underlying memory is not laid out contiguously

- contiguous creates a contiguous memory layout when necessary, allowing the following view operation to work intended

- operation - B - combine the head dim

- our setup:

$$B = 1, \quad T = 3, \quad d_{out} = 768$$

- we want to combine the final two dimensions:

$$(h, d_{head}) = (2, 384)$$

- after attention, each head produces context vectors of shape:

- $C_1, C_2 \in \mathbb{R}^{1 \times 3 \times 384}$

- $C = \text{Concat}(C_1, C_2)$

- $(1, 3, 384) + (1, 3, 384)$

$$= (1, 3, 768)$$

- Here + in the shape illustration means combining the feature dimensions, not adding the tensor values.

18) Attention output projection.

- after Concatenating our two attention heads, we need to combine their learned representation into a unified representation of each token.

- Input

$$C \in R^{1 \times 3 \times 768}$$

- operation - Learned Linear transformation

- we apply the output projection,

$$Y = C W_O^T + b_O$$

- where:
  - $W_O \in R^{768 \times 768}$, $b_O \in R^{768}$

- for each token, the dim are

$$(1 \times 768)\ (768 \times 768) \rightarrow (1 \times 768).$$

projected features

- The learned weight matrix allows each allows each output feature to depend on all 768 input features, including features from both attention heads.

- During training. the Model learns weights that determine how these features should be mixed

- output

$$Y \in R^{1 \times 3 \times 768}$$

19) Attention Dropout + First Residual connection

- Now we finish the multihead attention sublayer and connect it back to the original input.

- input:

$$Y \in R^{1 \times 3 \times 768}$$

$$X_{shortcut} \in R^{1 \times 3 \times 768}$$

→ operation - A - Apply dropout.

$$Y_{drop} = Dropout(Y)$$

$$Y_{drop} \in R^{1 \times 3 \times 768}$$

→ operation - B - Residual connection

- add the dropped-out attention output to the original input

$$X_{attn} = X_{shortcut} + Y_{drop}$$

- addition is element wise

$$(1, 3, 768) + (1, 3, 768) \rightarrow (1, 3, 768)$$

- Output

$$X_{attn} \in R^{1\times 3\times 768}$$

- attn sublayer is now complete.
- Second residual connection

$$S = X_{attn} \in R^{1\times 3\times 768}$$

2.8) Second Layer Normalization.

- as the attention Sublayer is complete. Now we Normalize it's output before sending it into FFN

- Input

$$X_{attn} \in R^{1\times 3\times 768}$$

— operation - Layer Normalization

- for each token

$$M_i = \frac{1}{768} \sum_{j=1}^{768} x_{i,j}$$

- the variance for each token

$$\sigma_i^2 = \frac{1}{768} \sum_{j=1}^{768} (x_{ij} - M_i)^2$$

- Normalize each feature.

$$x_{ij} = \frac{x_{ij} - M_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

- apply the Scale and shift parameters

$$y_{ij} = \gamma_j \cdot x_{ij} + \beta_j$$

- output :

$$X_{FFN-in} \in \mathbb{R}^{1 \times 3 \times 768}$$

2) FFN expansion :

$$d_{model} = 768$$

- we can choose any expansion factor r and cal

$$d_{FFN} = r \times d_{model}$$

- r doesn't have to be an integer. It can be fractional as long as $r \times 768$ gives a positive integer dimension.

- We'll keep $r = 4$.

- Input

$$x_{FFN-in} \in \mathbb{R}^{1 \times 3 \times 768}$$

→ operation – first linear transformation

$$d_{FF} = 4 \times 768 = 3072$$

- the FFN expands each token's feature representation from 768 to 3072 dim.

$$H = X_{FFN-in} W_1^T + b_1$$

- The weight matrix and bias have dimensions

$$W_1 \in R^{3072 \times 768}, \quad b_1 \in R^{3072}$$

- the dim calculation

$$(1, 3, 768) \xrightarrow{\text{Linear } 768 - 3072} (1, 3, 3072)$$

- each token is transformed independently using the same learned weights. the FFN does not mix information between token positions in this linear layer.

- output

$$H \in R^{1 \times 3 \times 3072}$$

22) GELU Activation

$$GELU = \frac{x}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715\, x^3\right)\right)\right]$$

- for our FNN, GELU is applied element-wise to all 3072 features of each token.

- we can represent

$$H_{GELU} = GELU(x)$$

- output

$$H_{GELU} \in \mathbb{R}^{1\times 3\times 3072}$$

- GELU introduces non linearity, allowing the FFN to learn more complex transformations than a stack of linear layer alone.

23) FNN Contraction (3072 → 768)

- we project each token's features back to the original model dimensions.

- input $H_{GELU} \in \mathbb{R}^{1\times 3\times 3072}$

→ Operation - Second Linear transformation

$$F = H_{GELU} W_2^T + b_2$$

- Where

$$W_2 \in R^{768 \times 3072}, \quad b_2 \in R^{768}$$

$$(1, 3, 3072) \longrightarrow (1, 3, 768)$$

- output

$$\boxed{F \in R^{1 \times 3 \times 768}}$$

- the learned transformation combines the 3072 intermidiate features into 768 output features for each token

2) FFN Dropout + Second Residual Connection.

- Now we complete our Single transformer block.

- Input

$$F \in R^{1 \times 3 \times 768}$$

- the Short cut saved before the second Layer Norm is the attention sub layer output

$$x_{attn} \in R^{1 \times 3 \times 768}$$

→ operation A - Dropout

$$F_{drop} = Dropout(F)$$

- Shape remains.

$$(1, 3, 768) \rightarrow (1, 3, 768).$$

→ operation B - Residual connection

- add the dropped-out FFN output to the saved shortcut

$$\boxed{X_{block} = X_{attn} + F_{drop}}$$

- element wise addition

$$(1, 3, 768) + (1, 3, 768) \rightarrow (1, 3, 768)$$

- output - Transformer block complete

$$\boxed{X_b \in \mathbb{R}^{1 \times 3 \times 768}}$$

25) Final layer Normalization

- we move outside the block to the final LayerNorm, which prepares the tokens representation for the output layer.

- input

$$X_{block} \in \mathbb{R}^{1 \times 3 \times 768}$$

→ operation - final LayerNorm

- for each token i, calculate its mean and variance across 768 features

$$M_i = \frac{1}{768} \sum_{j=1}^{768} x_{ij}$$

$$\sigma_i^2 = \frac{1}{768} \sum_{j=1}^{768} (x_{ij} - M_i)^2$$

- Normalize

$$x_{ij} = \frac{x_{ij} - M_i}{\sqrt{\sigma_i^2 + \epsilon}} \qquad \therefore \epsilon = 10^{-5}$$

- apply the scale and shift parameters.

$$y_{ij} = \gamma_j \cdot x_{ij} + \beta_j$$

- output

$$x_{final} \in R^{1 \times 3 \times 768}$$

26) Linear output layer.

- Now we are at the final learned layer of our GPT's forward pass: the vocabulary projection

- Input

$$X_{final} \in R^{1 \times 3 \times 768}$$

→ operation — linear projection

- we project each tokens 768 - dimension representation into a vector containing one score for every token in the vocabulary

$$Z = x_{final} \; W_{out}^{T} + b_{out}$$

- where:

$$W_{out} \in \mathbb{R}^{50257 \times 768}$$

$$b_{out} \in \mathbb{R}^{50257}$$

- the dim calculation is:

$$(1, 3, 768) \longrightarrow (1, 3, 50257)$$

- each of three positions receives 50,257 scores, one for every vocabulary token.

- output — logits

$$Z \in \mathbb{R}^{1 \times 3 \times 50257}$$

- these scores are called logits. they are not probabilities. they can be -ve, +ve or larger than 1

27% final vocabulary softmax.

- We're at the final probability conversion step. Our model has produced the logits for the last input token. Now we convert those scores into probabilities for the next token.

- Input - last position Logits

$$z_{last} \in \mathbb{R}^{1 \times 3 \times 50257}$$

- We select the logits for the last input position, because that position predicts the next token after "I love AI"

$$z_{last} = z[:, -1, :]$$

- The shape is

$$z_{last} \in \mathbb{R}^{1 \times 50257}$$

→ operation - Apply softmax

- for each vocabulary token i

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{50257} e^{z_j}}$$

- Softmax converts the logits into probabilities between 0 and 1, with all 50,257 probabilities summing to 1

- output

$$P \in R^{1 \times 50257}$$

→ operation - select the next token

- the model can select a token using greedy decoding

$$i^* = \arg\max_i P_i$$

- or sample a token according to the probability distribution.

28) Select and Decode the next Token

- Now we turn the predicted probability distribution into an actual token and append it to our input sequence.

→ operation - Decode the token ID, int text

- the tokenizer converts the selected token ID into its corresponding text fragment

$$t_{next} = Decode(i^*)$$

- Let's say the selected token might decode to "!" or "models".

→ operation - Append the new token

- original input was

$$S = [17, 245, 901]$$

- Suppose the selected token ID is $i^* = 120$.

$$S_{new} = [17, 245, 901, 120]$$

- The sequence length changes.

$$(1, 3) \rightarrow (1, 4)$$

- The model can now run another forward pass to predict the following token.

- output
- we have generated one additional token and

extended the input sequence

$$S_{new} = S \,||\, [i^*]$$

- Here || denotes concatination.

Generation is autoregressive. The model repeatedly predicts one token, appends it, and predicts again until a stopping condition is reached, such as an end-of-sequence token or the generation limit.

- This completes the next token generation step of forward pass